R7.6 run report: pooled rho across an isogeny class, and path-finding cost against component size

Run id `run-20261101T0039Z`. **Branch** `research/r7-6-r62-batch`, base `origin/main` 332716c307 (library `src/` unchanged; new example `examples/r7_6_r62_batch.rs`). **Host** a 4-vCPU shared VM (`ecbench host`: env class ECBENV3h13e37662a118), other build jobs running: **no isolation earned, no wall time is claimed**; every number below is an operation count. Toy instances only; nothing here concerns a deployed curve and no sub-rho claim is made. It finishes R6.2 (2230, corrected by 2543).

Verdicts (details and numbers in `RESULTS.md`)

- **H1 (pooling across a class of size C gives about a $\sqrt{C}$ gain over separate rho runs):** supported as a scaling law, falsified as the equality "gain $= \sqrt{C}$". The gain is above 1 with an interval excluding 1 at every $C \geq 4$ in all 7 runs; its exponent in C over 2..256 is 0.43 to 0.52 (ideal Kuhn–Struik: 0.489); at $C = 256$ the gain is 11.0 to 13.9 against 16, approaching the ideal 14.2 as r grows; the gain is below $\sqrt{C}$ by more than its interval from $C = 32$ in every run. Transported and same-curve pooling do not differ consistently (5 of 56 intervals exclude 1, mixed directions).
- **H2 (path-finding cost per pair is $O(\sqrt{C})$):** refuted for the corrected breadth-first bidirectional search: exponent 1.13 [1.06, 1.21] over 40 classes and 32 distinct component sizes from 8 to 16 702 curves (3.3 decades); 0.5 is outside every interval.
- **Limits:** toy sizes; ecbench arm is same-curve pooling at L0 with a same-host replay certificate, not an independent one, and no admitted benchmark.

What ran

- **Batch half** (native, `examples/r7_6_r62_batch.rs --mode batch`): one planted key on each of the first L curves of a breadth-first class of a base curve E_0 (class size $C = L$), transported to E_0 along the `SmallIsogeny::dual`-certified duals (an ℓ = 5, 7 edge enters the class only where `dual` certifies it; `--sym 1` draws $p \equiv \pm 1$ mod 5 and mod 7, `--sym 0` leaves p free). Arms on the library walk that ecbench's `rho.multitarget_pooled` and `rho.negation` run: **separate** (each transported target by its own `rho_walk_with`), **pooled B** (the first L targets in one `rho_batch_with`), **pooled C** (the control: L fresh keys on E_0, same pooled batch, same seed). Pooled B and separate share keys and seeds; the separate cost at L is the prefix sum. Every recovered log is verified on its original curve (181192 logs over 7 runs). Intervals: percentile bootstrap over trials, 10000 resamples.
- **ecbench half**: spec `spec.json` (`ECS1h0f7350931294`), session `ECBS1h4b0bd52ebc3a`, 640 records, all verified, `--cpus none` (L0, counts only), arms `rho.negation` and `rho.multitarget_pooled` with `batch` 4, 16, 64, 256, on registered `prime_search` curves of 20, 24, 28, 32 bits (16 workloads each, 2 measured rounds, no warm-up). Same-curve companions only (the control arm of the class experiment).
- **Path half** (native, `--mode scaling`, then `--mode fit`): whole certified components at p of 12 to 34 bits, random pairs by the corrected bidirectional search, routes re-validated forward; cost per pair = search enumerations + certifications + validation enumerations.

Audit and replay

`ecbench verify --replay-all --exit-code`: complete session, 640 records, 640 verified, 640 replays, all reproduced, problems: 0. Receipt `sessions/pooled-vs-rho-l0.audit.json`, SHA-256 `e120c0cb25583e61b40c14053070fc87240def172853a7b319d725114f783780`. The auditor ran on the same host class as the session, so this is a **same-host replay certificate, not an independent one**. The native example's records are replayed by `replay_check.py` (`replay_certificate.json`, below).

native record	SHA-256 (committed file)	replay
batch_r24_sym0_v1.json	300a1739ca6e9ba7...	identical except the wall-time note
batch_r24_sym1_v1.json	73b634586afd4d65...	identical except the wall-time note
batch_r28_sym0_v1.json	f7f06030c52b7b64...	identical except the wall-time note
batch_r28_sym1_v1.json	a9cc58d47020b060...	identical except the wall-time note
batch_r32_sym0_v1.json	a5701762ce392c5a...	identical except the wall-time note
batch_r32_sym1_v1.json	85bd38475f81a8d4...	identical except the wall-time note
batch_r36_sym1_v1.json	02d9c84f95e00c37...	identical except the wall-time note
scaling_fit_v1.json	9f42b74179338a05...	identical except the wall-time note
scaling_v1.json	e9b5a530158d9f0c...	identical except the wall-time note
scaling_v2.json	-	not replayed (skipped (time))

Replay binary SHA-256 0563025f75680351...; all replayed records identical: yes.

Batch half: gain of pooling over separate runs

Gain = $\sum$ separate / $\sum$ pooled B (counted group additions); strict prediction $\sqrt{L}$; the ideal Kuhn–Struik gain is $\frac{1}{\sum_{i<L}\binom{2i}{i}\frac{4^{-i}}{L}}$ (closed form, recorded by the example as `ks_prediction_ratio`).

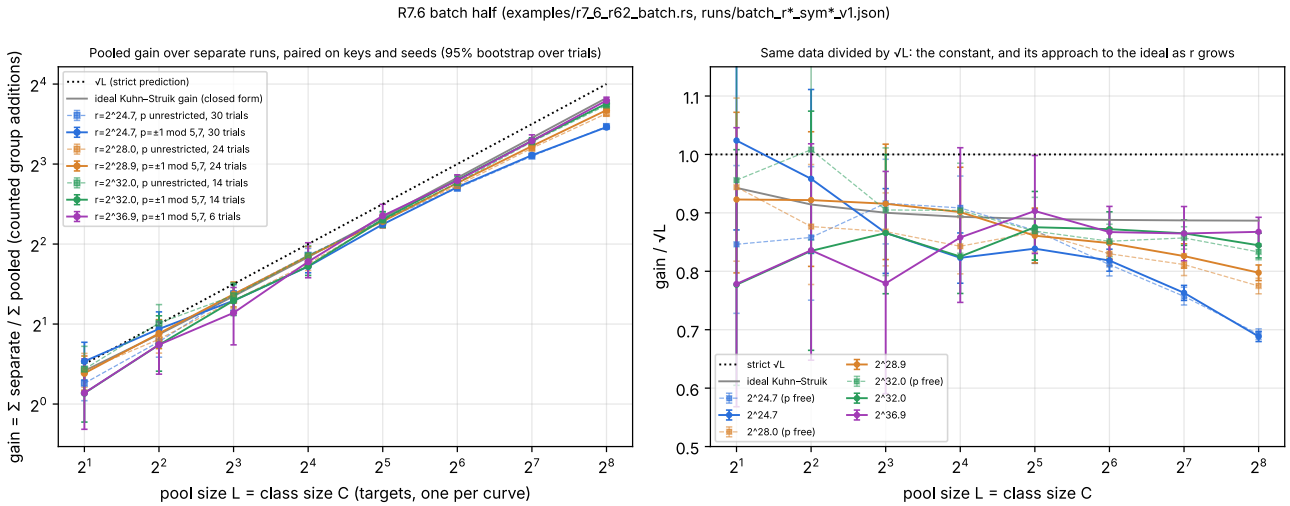


Figure 1: Recorded gain against $L = C$ for seven runs (95% bootstrap over trials), and the same divided by $\sqrt{L}$. Source `runs/batch_r*_sym*_v1.json`; script `plot.py`.

run	trials	gain at $L = 64$	gain at $L = 256$	gain / $\sqrt{L}$ at 256	pooled / separate over ideal KS at 256	gain exponent in L (2..256)
r=2^24.71, p free	30	6.49 [6.34, 6.65]	11.08 [10.93, 11.22]	0.692 [0.683, 0.701]	1.281 [1.264, 1.297]	0.458 [0.429, 0.488]
r=2^24.66, p=±1 mod 5,7	30	6.55 [6.4, 6.7]	11.01 [10.88, 11.15]	0.688 [0.68, 0.697]	1.288 [1.272, 1.304]	0.43 [0.396, 0.463]
r=2^28.04, p free	24	6.64 [6.48, 6.81]	12.4 [12.18, 12.61]	0.775 [0.761, 0.788]	1.144 [1.126, 1.164]	0.468 [0.442, 0.495]
r=2^28.88, p=±1 mod 5,7	24	6.79 [6.57, 6.99]	12.76 [12.55, 12.97]	0.798 [0.784, 0.811]	1.112 [1.094, 1.13]	0.468 [0.44, 0.498]
r=2^32.03, p free	14	6.81 [6.6, 7.04]	13.33 [13.12, 13.55]	0.833 [0.82, 0.847]	1.064 [1.047, 1.082]	0.466 [0.43, 0.506]
r=2^32.02, p=±1 mod 5,7	14	6.98 [6.77, 7.21]	13.51 [13.17, 13.84]	0.844 [0.823, 0.865]	1.05 [1.025, 1.078]	0.514 [0.461, 0.563]
r=2^36.86, p=±1 mod 5,7	6	6.94 [6.56, 7.29]	13.88 [13.49, 14.28]	0.868 [0.843, 0.892]	1.022 [0.994, 1.053]	0.522 [0.461, 0.59]

Predicted exponent 0.5 (strict) and 0.489 (the ideal Kuhn–Struik curve over $L = 2…256$). Pooled-cost exponents: 0.55, 0.57, 0.53, 0.53, 0.53, 0.5, 0.49 (predicted 0.5); separate-cost exponents: 1.01, 1, 0.99, 1, 1, 1.02, 1.02 (predicted 1).

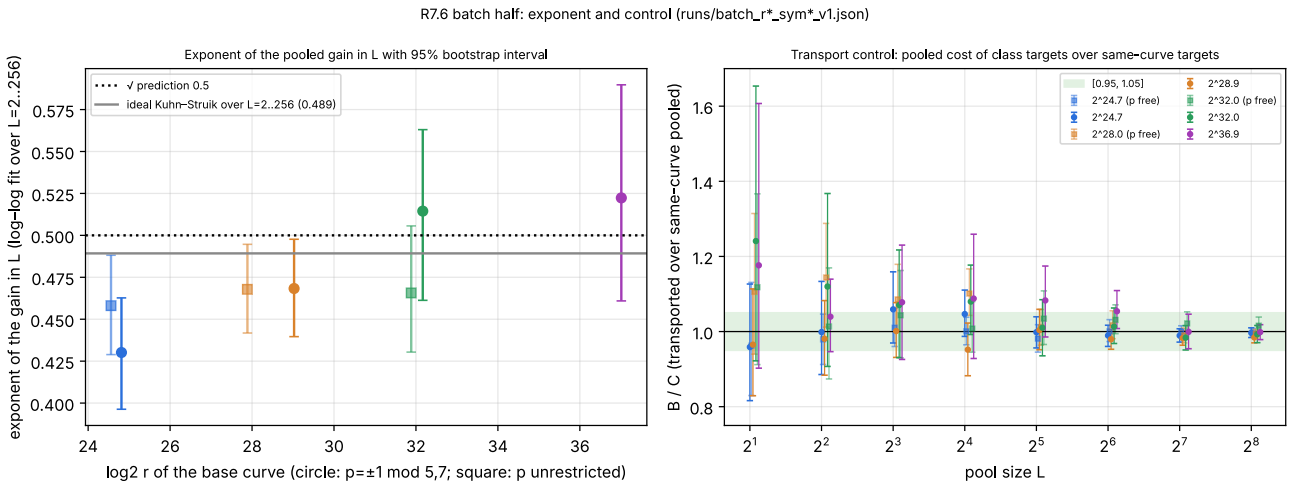


Figure 2: Left: exponent of the gain in L with 95% bootstrap interval, per run, against the strict 0.5 and the ideal Kuhn–Struik exponent. Right: transport control B/C per L (the interval and the band [0.95, 1.05]). Source `runs/batch_r*_sym*_v1.json`; script `plot.py`.

ecbench session (same-curve pooled arm, paired with `rho.negation`)

L	ecbench ratio pooled / `rho.negation` (cluster bootstrap)	pooled / separate = ratio $/L$ (derived)	gain $= L/$ ratio (derived)	per curve gain, $r = 2^{19.1}, 2^{24.0}, 2^{28.0}, 2^{31.7}$
4	2.31 [2.097, 2.549]	0.5775 [0.5243, 0.6372]	1.73 [1.57, 1.91]	1.66 ; 1.74 ; 1.66 ; 1.9
16	4.991 [4.549, 5.486]	0.3119 [0.2843, 0.3429]	3.21 [2.92, 3.52]	2.94 ; 3.25 ; 3.21 ; 3.59
64	11.354 [10.309, 12.524]	0.1774 [0.1611, 0.1957]	5.64 [5.11, 6.21]	4.61 ; 6 ; 5.93 ; 6.96
256	28.164 [24.968, 31.609]	0.11 [0.0975, 0.1235]	9.09 [8.1, 10.25]	6.35 ; 10.12 ; 10.9 ; 13.41

The separate cost is L times the mean single-run S of `rho.negation` on the same targets (one reference run per workload and round, 64 workloads, 128 pairs), so these intervals are wider than the example's, which runs every separate walk. Pricing band (unpriced `canonicalisations_uncharged`, priced 0, 0.1, 1 GAE): the order pooled > `rho.negation` in S holds at every weight at every L (recorded `ops.band.order`).

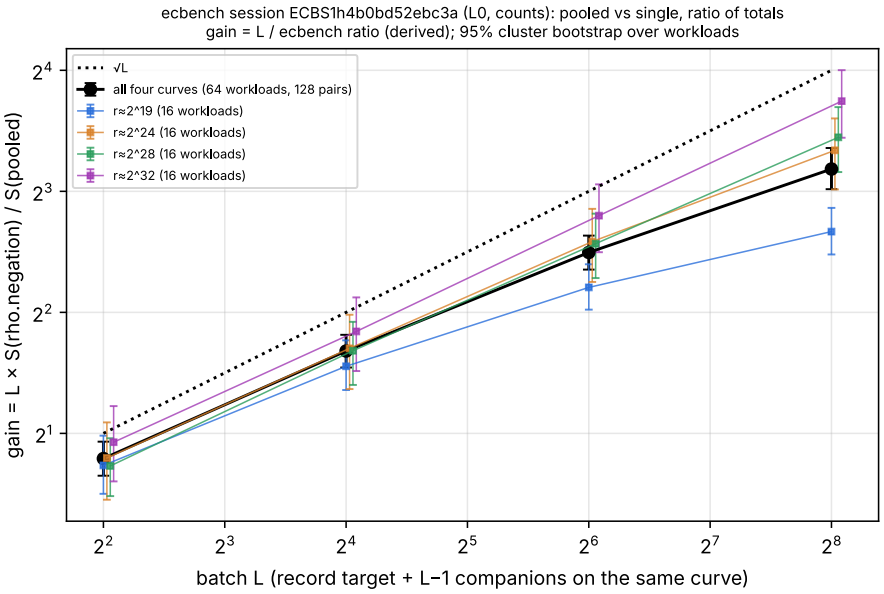


Figure 3: ECBS1h4b0bd52ebc3a: gain $= L/$ (ecbench ratio) with the cluster bootstrap interval, overall and per curve. Source `sessions/pooled-vs-rho-l0/comparisons/rho-neg__pooled-L*.json`; script `plot.py`.

Path half: cost against component size

subset	cost	classes	sizes	exponent (95% bootstrap)	contains 0.5	contains 1

all	total	40	8..16702	1.13 [1.059, 1.207]	no	no
all	search_only	40	8..16702	1.132 [1.054, 1.215]	no	no
size_ge_100	total	19	129..16702	1.085 [0.911, 1.297]	no	yes
size_ge_100	search_only	19	129..16702	1.075 [0.887, 1.304]	no	yes
size_ge_1000	total	11	1050..16702	1.276 [0.746, 1.904]	no	yes
size_ge_1000	search_only	11	1050..16702	1.308 [0.695, 2.041]	no	yes
size_lt_1000	total	29	8..728	1.159 [0.978, 1.284]	no	yes
size_lt_1000	search_only	29	8..728	1.167 [0.974, 1.3]	no	yes
ells_split_ge_3	total	17	8..16702	1.097 [0.954, 1.217]	no	yes
ells_split_ge_3	search_only	17	8..16702	1.094 [0.933, 1.223]	no	yes
ells_split_le_2	total	23	8..13760	1.16 [1.077, 1.269]	no	no
ells_split_le_2	search_only	23	8..13760	1.166 [1.079, 1.279]	no	no

40 classes, 32 distinct component sizes, 8 to 16702 curves (3.32 decades); every returned route re-validated forward (asserted in the run).

size bin	classes	mean size	mean total per pair	total / C (min..max)	total / $\sqrt{C}$
8..29	13	13	25	1.79 (0.95..3.23)	6.5
30..99	8	57	74	1.45 (0.35..4.28)	10.3
100..299	3	166	437	2.59 (0.79..4.1)	33.3
300..999	5	520	2026	3.87 (2.61..4.51)	87.9
1000..2999	5	1717	5429	3.01 (0.33..4.63)	127
3000..9999	3	5974	25324	4.43 (2.64..5.72)	332.6
10000..99999	3	13924	44439	3.29 (2.3..3.85)	381.1

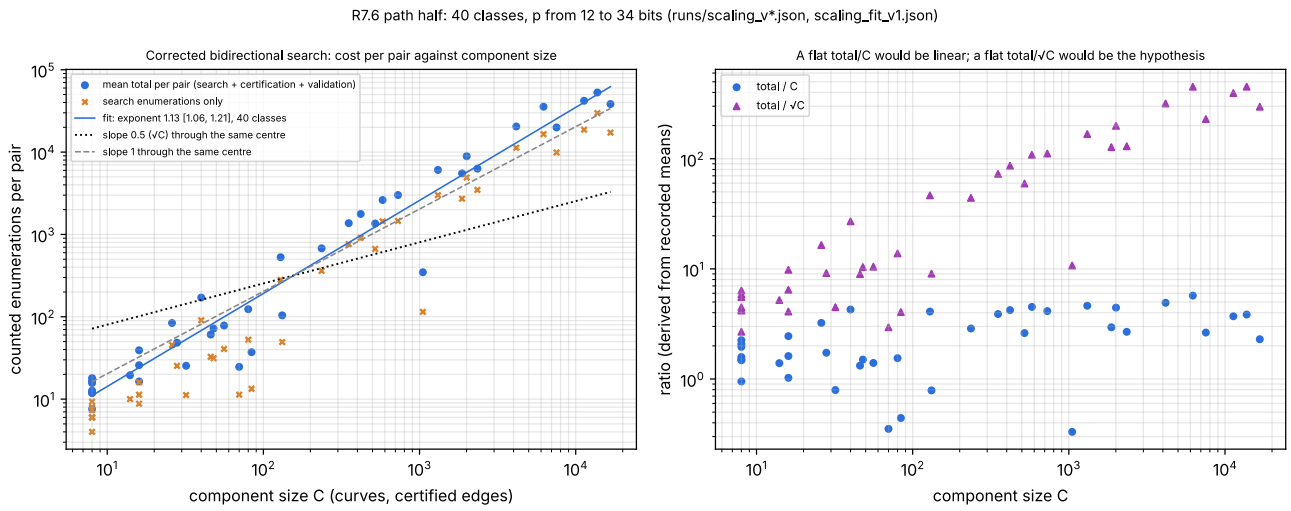


Figure 4: Recorded mean counted enumerations per pair against component size, one point per class (5 to 6 pairs each); fit and reference slopes through the same centre; right: the same divided by C and $\sqrt{C}$. Source `runs/scaling_v1.json`, `runs/scaling_v2.json`, `runs/scaling_fit_v1.json`; script `plot.py`.

Limits

- ecbench: same-curve pooled arm only (not the transported arm), L0 counts only, one host class, same-host receipt; the reference is one run per workload, so its intervals are wider than the example's. No isolated or independent-runner measurement, no EKS/isolab run, no admitted benchmark, no wall time.
- Batch half: the pooled class is the first $C \le 256$ curves of a breadth-first class, one base curve per run (seven base curves, r from $2^{24.7}$ to $2^{36.9}$), 6 to 30 trials; unregistered toy curves identified by p, a, b in the records. The distance from the ideal Kuhn–Struik cost at large C and small r is measured, its cause is not.

- Path half: 40 classes, 5 to 6 pairs each, components capped at 12 000 (v1) or 24 000 (v2) curves; only 11 classes exceed 1000 curves; the search is breadth-first bidirectional on a toy class, not an Elkies walk; `scaling_v2.json` was not replayed (33 minutes).
- Replay of the native records (`replay_certificate.json`): see `RESULTS.md`.